

TEMAS DE PROGRAMACION AVANZADA

SierraScan: Sistema Unisierra de Detección de Similitud Documental

RUBEN DARIO ANDRADE BARRERA

ISC 8 – 1

DR. JESÚS MIGUEL GARCÍA GORROSTIETA

Moctezuma, Sonora a 29 de mayo de 2026

ÍNDICE

INTRODUCCIÓN

La revisión de similitudes entre documentos académicos representa una tarea importante dentro de las instituciones educativas, ya que permite apoyar la detección de posibles coincidencias, copias o reutilización excesiva de contenido. Sin embargo, cuando esta revisión se realiza de forma manual, puede volverse lenta, repetitiva y propensa a errores, especialmente cuando se comparan varios archivos en formato PDF.

En respuesta a esta problemática, se desarrolló **SierraScan**, un sistema web orientado a la detección de similitud documental dentro del contexto académico de la Universidad de la Sierra. El sistema permite cargar documentos PDF, convertirlos a texto plano, procesar su contenido mediante técnicas de procesamiento de lenguaje natural y comparar los textos utilizando TF-IDF y similitud de coseno.

El propósito del proyecto es implementar un modelo funcional dentro de una página web, permitiendo al usuario visualizar porcentajes de similitud entre documentos y obtener una clasificación interpretativa del resultado. Además, se integró una prueba con dataset etiquetado para validar el comportamiento del sistema y calcular métricas como exactitud, precisión, recall y F1-score.

DESARROLLO

Conjunto de Datos Utilizado

Para validar el comportamiento del sistema se utilizó el dataset **Mharis205/MIT-PLAGAIRISM-DETECTION-DATASET**, disponible en Hugging Face. Este conjunto de datos contiene pares de textos etiquetados, donde cada registro permite comparar un texto A contra un texto B. La etiqueta 1 representa un par positivo o similar, mientras que la etiqueta 0 representa un par negativo o no similar.

Para validar el comportamiento inicial del sistema se utilizó el dataset *Mharis205/MIT-PLAGAIRISM-DETECTION-DATASET*, compuesto por pares de textos etiquetados como positivos o negativos. Durante la validación se identificó que el conjunto de datos contiene principalmente oraciones cortas, por lo que sus resultados se interpretaron como una prueba inicial del algoritmo y no como una representación completa de documentos académicos extensos. Aun así, el dataset permitió comprobar el flujo de procesamiento, comparación y evaluación mediante métricas como exactitud, precisión, recall y F1-score.

Para complementar esta limitación, también se realizaron pruebas con documentos PDF de mayor extensión, comparando un documento original, una versión parafraseada y un documento de tema distinto. En esta prueba, el sistema obtuvo 74.22% de similitud entre los documentos relacionados y valores cercanos al 10% con documentos no relacionados, lo que demuestra que el sistema puede distinguir adecuadamente entre textos similares y textos de contenido diferente.

Elemento	Descripción
Dataset	Mharis205/MIT-PLAGAIRISM-DETECTION-DATASET
Fuente	Hugging Face
Tipo de datos	Pares de textos etiquetados
Clase 0	Par no similar / no plagio
Clase 1	Par positivo / similar
Uso en el proyecto	Validación del modelo

Etapas de Preprocesamiento y Limpieza Textual

Antes de realizar la comparación textual, el sistema aplica una etapa de preprocesamiento. En primer lugar, si el archivo cargado es PDF, se extrae su contenido mediante la **librería PyMuPDF**, convirtiendo el documento a texto plano.

Posteriormente, el texto se normaliza para reducir variaciones innecesarias y facilitar el análisis.

El flujo de preprocesamiento aplicado fue el siguiente:

1. Extracción de texto plano desde archivos PDF.
2. Conversión del texto a minúsculas.
3. Eliminación de acentos.
4. Eliminación de enlaces, símbolos y caracteres especiales.
5. Tokenización del texto en palabras.
6. Eliminación de stopwords.
7. Reconstrucción del texto limpio para su comparación.

Este proceso permite reducir ruido textual y conservar términos más representativos del contenido, mejorando la comparación entre documentos mediante técnicas de vectorización.

Algoritmo de Aprendizaje e Implementación

El núcleo del sistema se basa en un **modelo de similitud textual** compuesto por dos etapas principales: la vectorización mediante **TF-IDF** y el cálculo de **similitud de coseno**.

La técnica TF-IDF transforma cada documento en un vector numérico, asignando mayor peso a los términos más relevantes y menor peso a las palabras comunes. Posteriormente, la similitud de coseno compara los vectores generados para determinar qué tan parecidos son dos documentos. El resultado obtenido se expresa como un valor entre 0 y 1, el cual se transforma a porcentaje para facilitar su interpretación en la interfaz web.

Aunque no se utilizó un clasificador supervisado tradicional como KNN, árbol de decisión o SVM, el sistema implementa una **clasificación basada en umbrales**. De esta forma, el porcentaje de similitud se interpreta en tres niveles: similitud **baja**, similitud **sospechosa** y posible **plagio**.

Rango de similitud	Clasificación
0% - 44.99%	Similitud baja
45% - 74.99%	Similitud sospechosa
75% - 100%	Posible plagio

Algoritmo Implementado: TF-IDF y Similitud de Coseno

Para este proyecto se seleccionó **TF-IDF** combinado con **similitud de coseno**, debido a que permite comparar documentos completos de manera sencilla, rápida e interpretable. Aunque existen otros métodos como **Levenshtein**, **Jaccard** o **LCS**, estos se enfocan más en comparar cadenas, conjuntos de palabras o secuencias específicas, mientras que TF-IDF permite representar documentos completos como vectores numéricos.

Técnica	Descripción	Uso en el proyecto
Levenshtein	Calcula ediciones entre textos	Considerada
Jaccard	Compara intersección de palabras	Considerada
LCS	Busca secuencias compartidas	Considerada
WCO	Compara bloques de palabras	Referencia teórica
TF-IDF + Coseno	Compara vectores textuales	Implementada

Desempeño Alcanzado por el Modelo

Para evaluar el comportamiento del sistema se utilizó una muestra de 200 pares de textos del dataset. Cada par fue procesado mediante normalización, tokenización, filtrado de stopwords, vectorización TF-IDF y similitud de coseno. Posteriormente, la predicción generada por el sistema fue comparada contra la etiqueta real del dataset.

Para esta evaluación se utilizó un umbral de clasificación del **17%**, con el cual el sistema obtuvo una exactitud global de **65%**.

Tabla global

Métrica	Resultado
Pares evaluados	200
Umbral de evaluación	17%
Exactitud global	65%

La exactitud global indica el porcentaje total de predicciones correctas realizadas por el sistema durante la evaluación.

Tabla por clase

Clase	Precisión	Recall	F1-score
Plagio	63.64%	70%	66.67%
No plagio	—	—	63.16%

En la clase “Plagio”, el sistema obtuvo una precisión de **63.64%**, lo que indica que una parte considerable de los pares marcados como positivos fueron correctos. Además, alcanzó un recall de **70%**, lo que significa que logró identificar la mayoría de los pares positivos del dataset. El F1-score de **66.67%** resume el equilibrio entre precisión y recall.

Estos resultados muestran un desempeño aceptable para un modelo basado en TF-IDF y similitud de coseno. Sin embargo, debido a que el dataset contiene principalmente oraciones cortas, los resultados se consideran una validación inicial del algoritmo y no una representación completa del comportamiento del sistema en documentos académicos extensos.

INTERFAZ GRÁFICA DEL SISTEMA

Vista General de la Interfaz Web

La interfaz gráfica fue desarrollada con React y Vite, manteniendo una apariencia limpia e institucional inspirada en herramientas de detección documental. La pantalla principal presenta el nombre del sistema, los controles de carga, el área para arrastrar documentos PDF, la lista de archivos almacenados y la tabla de resultados.

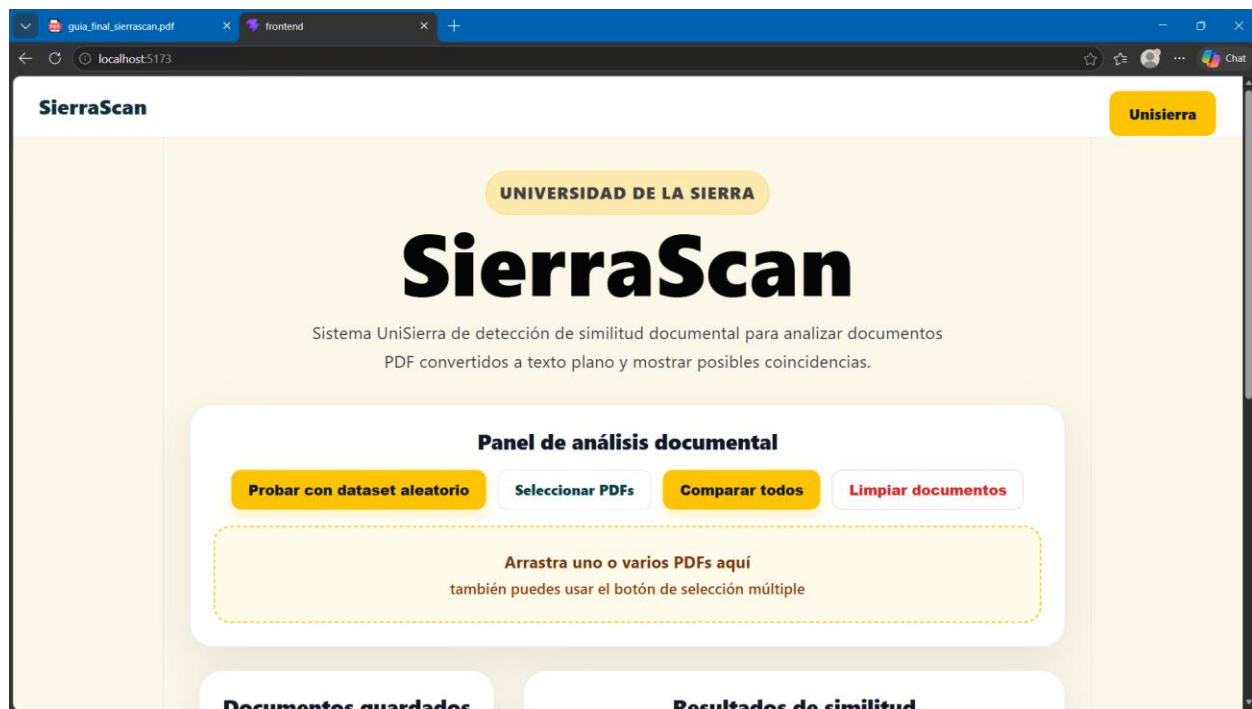


Figura 1. Vista general de la interfaz web SierraScan.

Carga de Documentos PDF

El sistema permite cargar uno o varios documentos PDF mediante selección manual o arrastrando los archivos al área indicada. Una vez seleccionados, los documentos se envían automáticamente al backend, donde se extrae su texto, se procesa y se almacena en la base de datos.

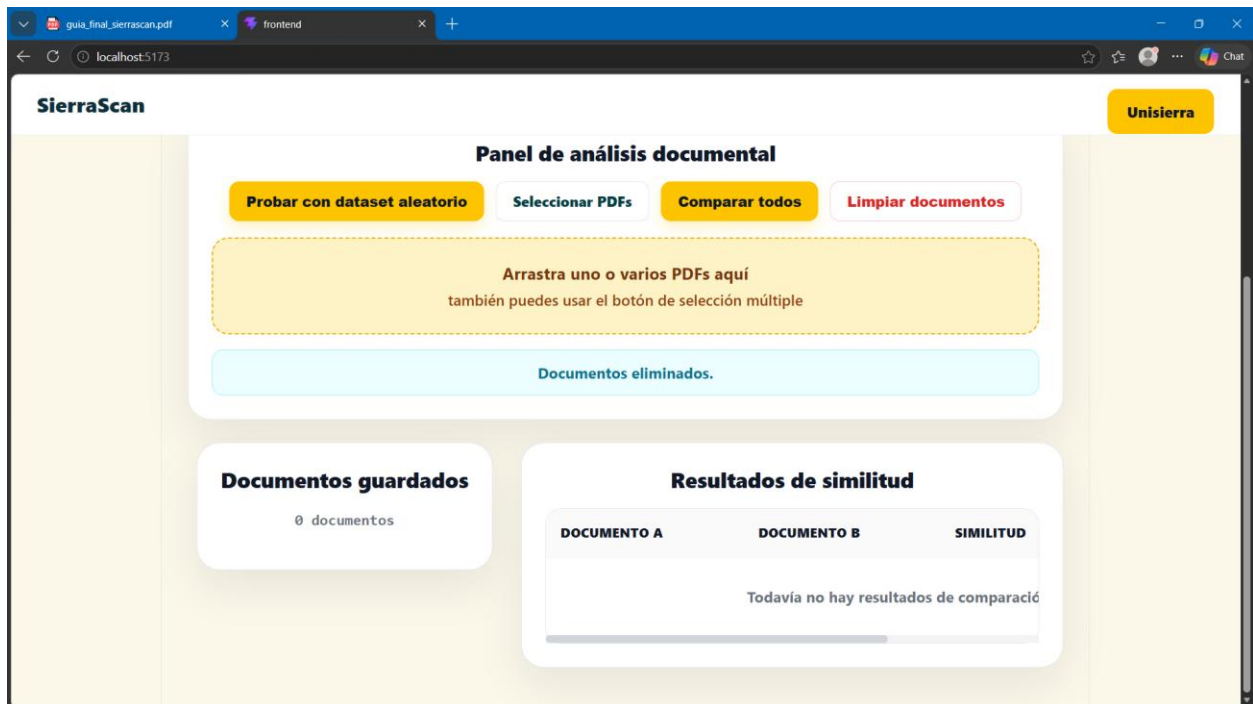


Figura 2. Carga múltiple de documentos PDF.

Prueba de Comparación entre Documentos

Para validar el funcionamiento con documentos PDF, se utilizaron tres archivos de prueba: un documento original, una versión parafraseada y un documento de tema distinto. Al realizar la comparación, el sistema detectó una similitud de 74.22% entre el documento original y la versión parafraseada, clasificándola como similitud sospechosa. Por otro lado, las comparaciones con el documento de tema distinto obtuvieron valores de 10.55% y 9.67%, clasificándose como similitud baja.

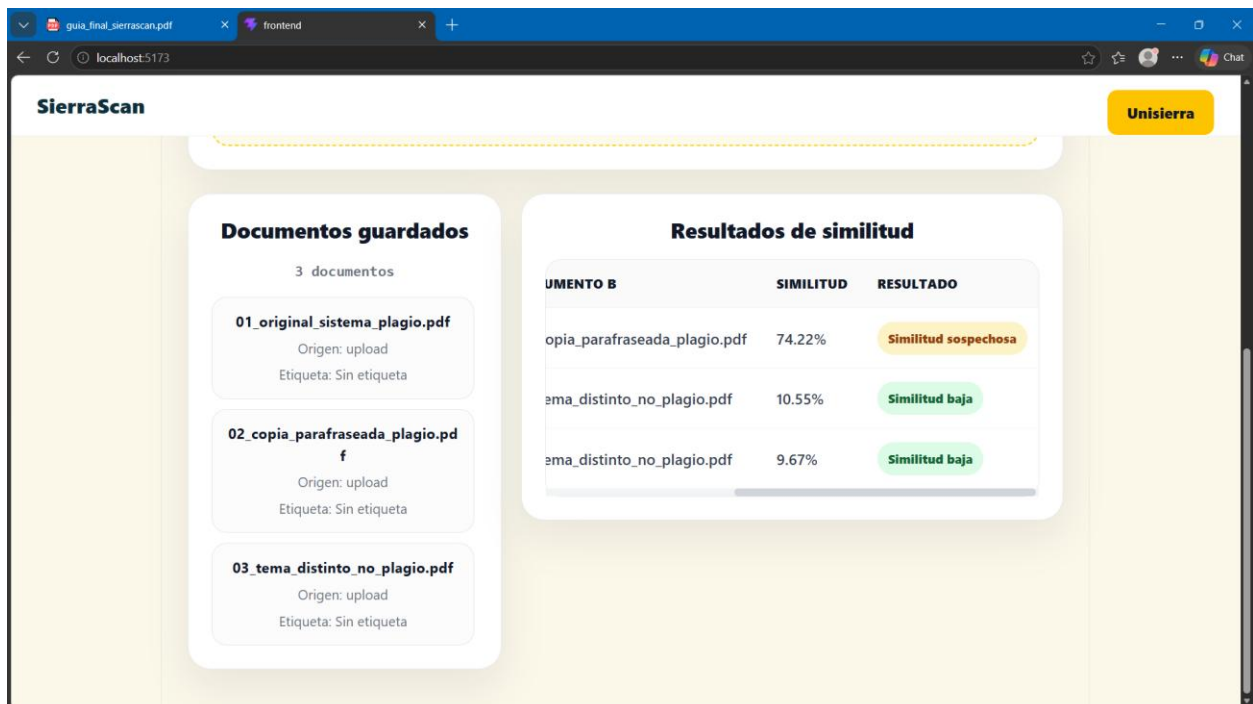
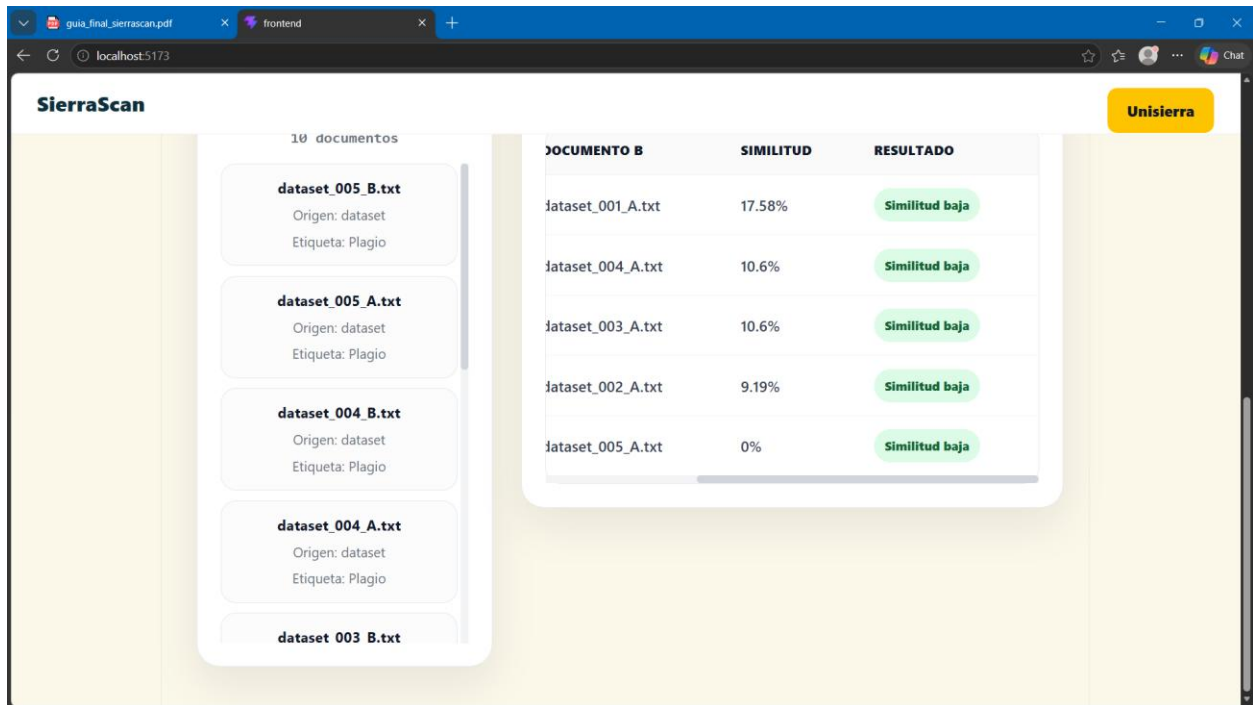


Figura 3. Resultados de comparación entre documentos PDF.

Prueba con Dataset Aleatorio

Además de la carga manual de PDFs, se agregó un botón para ejecutar una prueba automática con dataset aleatorio. Esta función selecciona un conjunto de pares de textos, los procesa, los guarda temporalmente y ejecuta la comparación, permitiendo comprobar el comportamiento del sistema sin depender de archivos externos.



The screenshot shows the SierraScan web application interface. On the left, there is a list of 10 documents, with the first five visible: dataset_005_B.txt, dataset_005_A.txt, dataset_004_B.txt, dataset_004_A.txt, and dataset_003_B.txt. Each document entry shows its origin as 'dataset' and its label as 'Plagio'. On the right, a table displays the comparison results for these documents. The table has three columns: DOCUMENTO B, SIMILITUD, and RESULTADO. The results show similarity percentages ranging from 0% to 17.58%, all labeled as 'Similitud baja' (Low Similarity).

DOCUMENTO B	SIMILITUD	RESULTADO
dataset_001_A.txt	17.58%	Similitud baja
dataset_004_A.txt	10.6%	Similitud baja
dataset_003_A.txt	10.6%	Similitud baja
dataset_002_A.txt	9.19%	Similitud baja
dataset_005_A.txt	0%	Similitud baja

Figura 4. Prueba automática con dataset aleatorio.

Documentación de la API

El backend del sistema cuenta con documentación automática generada por FastAPI. Desde esta sección es posible probar endpoints como la carga de documentos, comparación de textos, carga de dataset y evaluación del modelo.

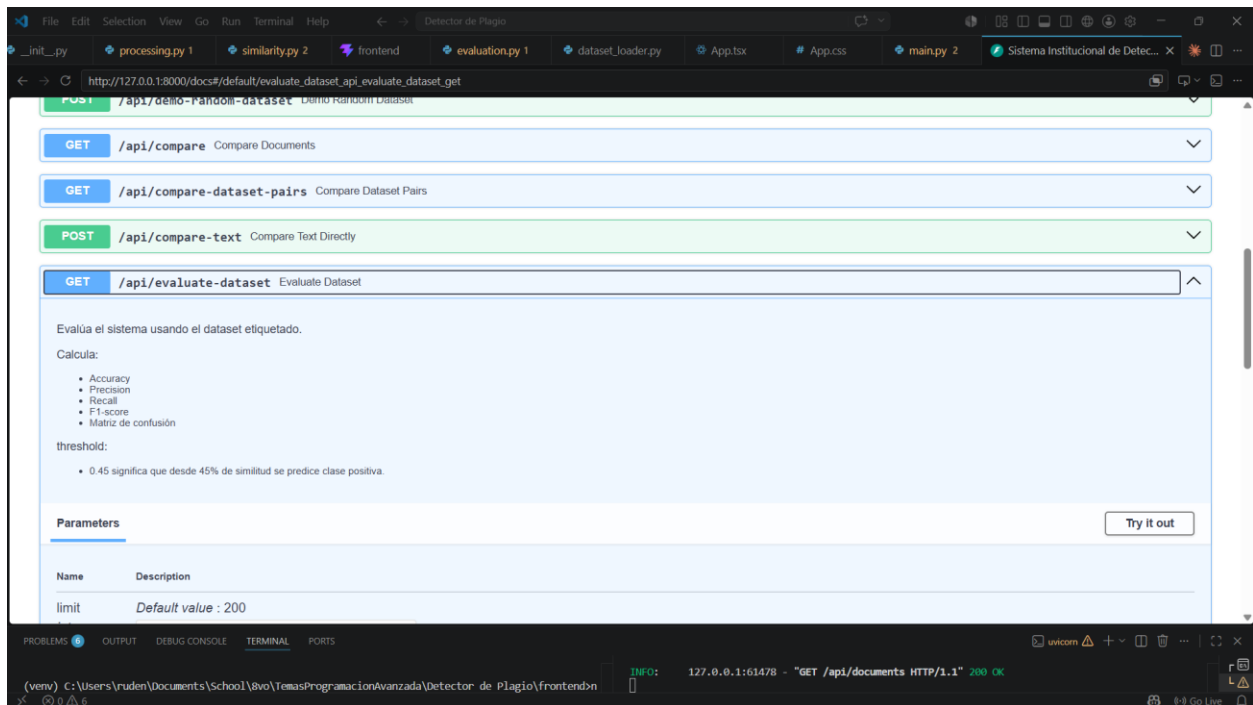


Figura 5. Documentación automática del backend en FastAPI.

EXPLICACIÓN GENERAL DEL CÓDIGO

El proyecto fue organizado en módulos para separar responsabilidades y facilitar su mantenimiento. El backend contiene archivos dedicados al almacenamiento, procesamiento textual, carga del dataset, comparación de documentos y evaluación del modelo. Por su parte, el frontend contiene la interfaz de usuario y las funciones encargadas de comunicarse con la API.

Tabla:

Archivo	Función
main.py	Define endpoints del backend
processing.py	Extrae, normaliza, tokeniza y filtra texto
similarity.py	Calcula TF-IDF, similitud de coseno y clasificación
dataset_loader.py	Carga ejemplos del dataset
evaluation.py	Calcula accuracy, precision, recall y F1
storage.py	Guarda documentos en SQLite
App.tsx	Interfaz web y conexión con la API

VIDEO DEMOSTRATIVO

Como complemento del documento, se elaboró un video demostrativo publicado en YouTube en modo no listado. En el video se explica el objetivo del sistema, el dataset utilizado, el modelo implementado, las métricas obtenidas, el funcionamiento de la página web y una revisión general del código principal.

Enlace al video:

Enlace al repositorio GitHub:

CONCLUSIONES Y RECOMENDACIONES

Dificultades Encontradas durante el Desarrollo

Durante el desarrollo del proyecto se presentaron diferentes dificultades. Una de las principales fue comprender la diferencia entre un clasificador supervisado tradicional y un modelo de similitud textual basado en umbrales.

También fue necesario ajustar el manejo del dataset y los umbrales de similitud. Al tratarse de pares de oraciones cortas, algunos resultados variaban bastante, por lo que se realizaron pruebas de evaluación para obtener métricas más equilibradas y comprender mejor el comportamiento del modelo TF-IDF.

Recomendaciones

Redacción:

Para futuras versiones se recomienda mejorar el sistema mediante pruebas con documentos académicos más extensos y variados, ya que el dataset utilizado contiene principalmente pares de oraciones cortas. También sería conveniente agregar la generación automática de reportes en PDF, comparación por secciones del documento, almacenamiento histórico de análisis realizados y una visualización más detallada de los fragmentos con mayor similitud. Estas mejoras permitirían que el docente no solo observe un porcentaje general, sino también identifique con mayor precisión qué partes del texto presentan coincidencias relevantes.